

Министерство науки и высшего образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №5

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Определение характеристик графов»

Выполнили:

студенты групп 24ВВВЗ

Масюк А. Р.

Мамонтов Н. П.

Приняли:

Юрова О. В.

Деев М. В.

Пенза 2025

Цель работы

Освоить на практике методы анализа характеристик графов с использованием матриц смежности на языке C. Закрепить навыки определения основных свойств графов: размера графа, степеней вершин, поиска изолированных, концевых и доминирующих вершин. Изучить принципы представления графов в памяти и их анализа с использованием матричных методов.

Лабораторное задание

Задание:

1. Сгенерировать (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.
2. Определить размер графа G, используя матрицу смежности графа.
3. Найти изолированные, концевые и доминирующие вершины.

Работа программы

```
Консоль отладки Microsoft Visual Studio
Введите количество вершин графа: 5

1. Матрица смежности графа G:
0 0 0 0 0
0 0 1 0 0
0 1 0 0 0
0 0 0 0 0
0 0 0 0 0
```

Матрица смежности графа G

```
2. Размер графа G (количество ребер): 1
```

Определение размера графа

```
3. Степени вершин: 0, 1, 1, 0, 0
   Изолированные вершины: 0, 3, 4
   Концевые вершины: 1, 2
   Доминирующие вершины: нет
```

Нахождение вершин

Вывод

Освоили на практике методы анализа характеристик графов с использованием матриц смежности на языке C. Закрепили навыки работы с

графами: определение размера графа, вычисление степеней вершин, поиск изолированных, концевых и доминирующих вершин. Реализовали генерацию случайных графов и их визуализацию через матричное представление. Написали рабочий код для полного анализа свойств графов согласно теоретическим основам.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

int random_int(int min, int max) {
    return min + rand() % (max - min + 1);
}

// 1. Генерация матрицы
void generateMatrix(int n, float p, int** matrix) {
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            if (i == j) {
                matrix[i][j] = 0;
            }
            else {
                float r = (float)rand() / RAND_MAX;
                if (r < p) {
                    matrix[i][j] = 1;
                }
                else {
                    matrix[i][j] = 0;
                }
                matrix[j][i] = matrix[i][j];
            }
        }
    }
}

// 2. Определение размера графа
int graphSize(int n, int** matrix) {
    int edges = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (matrix[i][j] == 1) {
                edges++;
            }
        }
    }
    return edges;
}

// 3. Поиск изолированных, концевых и доминирующих вершин
void findVertices(int n, int** matrix, int* degrees,
    int* isolated, int* isolated_count,
    int* pendant, int* pendant_count,
    int* dominating, int* dominating_count) {
    for (int i = 0; i < n; i++) {
        int deg = 0;
        for (int j = 0; j < n; j++) {
            deg += matrix[i][j];
        }
        degrees[i] = deg;

        if (deg == 0) {
            isolated[*isolated_count] = i;
            (*isolated_count)++;
        }
        else if (deg == 1) {
            pendant[*pendant_count] = i;
            (*pendant_count)++;
        }
    }
}
```

```

    }

    if (deg == n - 1) {
        dominating[*dominating_count] = i;
        (*dominating_count)++;
    }
}

// Вывод матрицы
void printMatrix(int n, int** matrix) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

// Вывод массива
void printArray(int arr[], int count, const char* label) {
    printf("%s: ", label);
    if (count == 0) {
        printf("Нет");
    }
    else {
        for (int i = 0; i < count; i++) {
            printf("%d", arr[i]);
            if (i < count - 1) printf(", ");
        }
    }
    printf("\n");
}

int main() {
    setlocale(LC_ALL, "RUS");
    srand(time(NULL));

    int n;
    float p = 0.3;

    printf("Введите количество вершин графа: ");
    scanf("%d", &n);

    int** matrix = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        matrix[i] = (int*)malloc(n * sizeof(int));
    }

    int* degrees = (int*)malloc(n * sizeof(int));
    int* isolated = (int*)malloc(n * sizeof(int));
    int* pendant = (int*)malloc(n * sizeof(int));
    int* dominating = (int*)malloc(n * sizeof(int));

    int isolated_count = 0;
    int pendant_count = 0;
    int dominating_count = 0;

    generateMatrix(n, p, matrix);

    printf("\n1. Матрица смежности графа G:\n");
    printMatrix(n, matrix);

    int size = graphSize(n, matrix);
    printf("\n2. Размер графа G (количество ребер): %d\n", size);

    findVertices(n, matrix, degrees,
        isolated, &isolated_count,
        pendant, &pendant_count,
        dominating, &dominating_count);

    printf("\n3. Степени вершин: ");
    for (int i = 0; i < n; i++) {
        printf("%d", degrees[i]);
        if (i < n - 1) printf(", ");
    }
    printf("\n");
}

```

```
printArray(isolated, isolated_count, "    Изолированные вершины");
printArray(pendant, pendant_count, "    Концевые вершины");
printArray(dominating, dominating_count, "    Доминирующие вершины");

for (int i = 0; i < n; i++) {
    free(matrix[i]);
}
free(matrix);
free(degrees);
free(isolated);
free(pendant);
free(dominating);

return 0;
}
```